# rust64 User Guide

Rust for the Commodore 64

Codegen backend, the c64 library and a cookbook

rust64 compiles Rust programs for the Commodore 64. You write real Rust (`no_std`, traits, generics, closures, iterators, `Option`, `match`), with the usual tools (rust-analyzer, clippy, `cargo test` for the logic), and rust64 turns it into a program that the C64 loads and runs. It is a codegen backend for rustc: rustc does parsing, type checking, borrow checking and monomorphization, and rust64 translates the resulting MIR into 6510 assembly, which rat assembles into a `.prg` and a `.d64`. The `c64` crate gives you the machine: text, sprites, bitmap graphics, sound, raster interrupts, joysticks, disk drives.

```
#![no_std]
#![no_main]

use c64::hw::{self, Color};

#[c64::main]
fn main() {
    c64::println!("hello from rust64");
    hw::vic().border.write(Color::Green);
}
```

```
$ cargo c64 build --release
    wrote .../target/mos-c64/release/hello.prg
```

This guide explains how to install rust64, which part of Rust it supports, how the generated program uses the machine, and how to use the `c64` library. Chapter 9 is a cookbook with complete examples.

**Contents**


1. Installation
2. Your first program
3. The cargo c64 command
4. Rust on the C64
5. Memory and the runtime
6. Attributes and macros
7. Assembly functions
8. Library reference
9. Cookbook
10. Inside the backend
11. Testing
12. Troubleshooting
13. Limitations and roadmap

# 1. Installation

| Tool | Purpose | Where rust64 looks |
|---|---|---|
| rustup | installs the pinned nightly toolchain | `rust-toolchain.toml` |
| **rat** (source checkout) | assembler, linked into the backend as a library | `../rat` next to this repository |
| **mimic** (optional) | runs programs (`cargo c64 run`) | `RUST64_MIMIC`, else `mimic` on the `PATH` |
| **mimic-headless** (optional) | end-to-end tests | `RUST64_MIMIC_HEADLESS`, else `../mimic/target/release` |
| C64 ROMs (for mimic) | KERNAL, BASIC, character ROM | `EMU_ROM_DIR`, else `~/.mimic/roms` |

rust64 needs a fixed nightly, because a codegen backend uses rustc's internal crates. `rust-toolchain.toml` pins it together with the components `rustc-dev`, `rust-src` and `llvm-tools`; rustup installs them on the first build. The repositories are expected side by side:

```
git/
  c64_rust/      this repository
  rat/           the assembler
  mimic/         the emulator (optional)
```

Build the backend and the cargo subcommand:

```
cd c64_rust
cargo build --release -p rustc_codegen_rust64 -p cargo-c64
```

This puts `cargo-c64` and the backend (`rustc_codegen_rust64.dll`, `.so` or `.dylib`) side by side in `target/release`. `cargo-c64` finds the backend next to itself; set `RUST64_BACKEND` to use another one. Put `target/release` on your `PATH` so that `cargo c64` works everywhere:

```
set PATH=%PATH%;C:\Users\me\git\c64_rust\target\release       (Windows)
export PATH=$PATH:$HOME/git/c64_rust/target/release           (Linux, macOS)
```

## A project

A rust64 program is a normal cargo package with a binary target that depends on the `c64` crate:

```
mygame/
  Cargo.toml
  src/
    main.rs
    sprites.rat        (optional: assembly)
  music.sid            (optional: a SID tune)
```

```toml
[package]
name = "mygame"
version = "0.1.0"
edition = "2024"

[dependencies]
c64 = { path = "../c64_rust/lib/c64" }

[profile.dev]
opt-level = 1

[profile.release]
opt-level = "s"
```

`opt-level = 1` in the dev profile is recommended: it lets rustc's MIR inliner remove the small helper calls that iterators and `core` produce. The release profile with `"s"` gives the smallest programs.

The `c64` crate also builds on the host (without `cargo c64`): then it uses stand-ins, a simulated 64 KB memory and stdout for printing, so rust-analyzer, clippy and `cargo test` work as usual.

## 2. Your first program

`src/main.rs`:

```
//! Hello: prints a greeting and the first Fibonacci numbers, then turns the
//! border green.
#![no_std]
#![no_main]

use c64::hw::{self, Color};

fn fib(n: u8) -> u16 {
    let (mut a, mut b) = (0u16, 1u16);
    for _ in 0..n {
        (a, b) = (b, a + b);
    }
    a
}

#[c64::main]
fn main() {
    c64::println!("hello from rust64");
    for i in 0..10 {
        c64::print!(fib(i), " ");
    }
    c64::println!();
    hw::vic().border.write(Color::Green);
}
```

- `#![no_std]`: the C64 has no operating system for `std`; `core` is available completely.
- `#![no_main]` and `#[c64::main]`: the runtime starts the function marked with `#[c64::main]`. When it returns, the program returns to BASIC.
- `c64::println!` takes a comma-separated list of values (strings, numbers, characters, `bool`, see c64::fmt). It does not use `core::fmt`, which would cost kilobytes.
- There is no `#[panic_handler]` in the program: the `c64` crate brings it.

Build and run:

```
cargo c64 build --release          # target/mos-c64/release/hello.prg
cargo c64 run --release            # build and start mimic
```

The program prints

```
HELLO FROM RUST64
0 1 1 2 3 5 8 13 21 34
```

and the border turns green. (Strings are ASCII in the source and are converted to PETSCII: in the C64's default character set both upper and lower case letters show as capitals.)

The repository has more examples in `examples/`: `hello`, `sprites`, `drawing`, `music`, `dir` (the same as c64_go's), `joystick`, `scroller` (a horizontal soft scroll: `$D016` set by two raster interrupts for one text row), `diagonal` (the whole screen scrolls diagonally over a pattern in a custom character set, with two screens that are swapped) and `border` (the smallest program, without the library).

## 3. The cargo c64 command

```
cargo c64 build [--release] [-o FILE.prg|FILE.d64] [cargo args...]
cargo c64 run   [--release] [-o FILE.prg|FILE.d64] [cargo args...]
cargo c64 asm   [--release] [cargo args...]
```

| Command | Does |
|---|---|
| `build` | builds the program: `target/mos-c64/<profile>/<name>.prg` |
| `run` | builds and starts mimic (`--prg`, or `--disk` for a `.d64`) |
| `asm` | builds and prints the generated assembly listing |
| `-o FILE` | also copies the result to FILE; with `.d64` the disk image |
| `--release` | the release profile |

Other arguments go to `cargo build` (for example `-p`, `--features`, `-v`).

Behind the scenes `cargo c64` writes the target specification (`target/rust64/mos-c64.json`) and runs

```
cargo build -Zbuild-std=core -Zjson-target-spec --target target/rust64/mos-c64.json
  with RUSTFLAGS = -Zcodegen-backend=<backend> -Zalways-encode-mir -Zinline-mir
```

with the pinned nightly. `core` is built from source for the C64 target. Library crates (`core`, `c64`, your own) produce no code, only their MIR; when rustc compiles the program, the backend translates everything that `main` reaches. Unused library code costs nothing.

Every build writes three files. The `.prg` is copied to `target/mos-c64/<profile>/`; the assembly listing (`.rat`) and the disk image (`.d64`, with the program as its only file) are next to cargo's build files in `target/mos-c64/<profile>/build/<package>/<hash>/out/`. `cargo c64 asm` and `-o x.d64` find them for you.

When the backend changes, `cargo c64` cleans the program's crate, because cargo cannot see that change itself.

| Variable | Meaning |
|---|---|
| `RUST64_BACKEND` | path of the backend library |
| `RUST64_MIMIC` | the emulator for `cargo c64 run` |
| `RUST64_MIMIC_HEADLESS` | the headless emulator for the tests |
| `RUST64_TRACE` | print the MIR of every compiled function |

## 4. Rust on the C64

### What works

- All of `core` that does not need floats or the heap: `Option`, `Result`, `?`, slices and their methods (`iter`, `get`, `split_at`, `chunks`, `contains`, ...), `&str` and byte strings, ranges, arrays, `core::mem`, `core::ptr`, `core::hint::black_box`.
- `struct`, `enum` (also with data), tuples, `match` with every pattern, `if let`, `let else`, `while let`, labelled `break` / `continue`.
- Generics, traits and trait methods (resolved at compile time), associated constants, const generics, `impl Trait`.
- Closures (capturing by reference or by value) and every iterator adapter: `map`, `filter`, `enumerate`, `zip`, `rev`, `step_by`, `sum`, `fold`, `any`, `position`, ...
- `static` and `static mut`, `const`, `const fn` (evaluated by rustc: tables cost no code).
- Function pointers (`fn(u8) -> u16`), also non-capturing closures as function pointers.
- `Drop`: values are dropped at the end of their scope (a `dos::File` closes itself).
- Integer arithmetic of every width with overflow checks in debug builds, `wrapping_*`, `checked_*`, `saturating_*`, shifts, rotations, `count_ones`, `leading_zeros`, ...
- Panics: bounds checks, `unwrap` on `None`, overflow in debug builds, `unreachable!()`, `assert!`.

### Sizes

| Type | Bytes | Note |
|---|---|---|
| `u8`, `i8`, `bool` | 1 | native: prefer them |
| `u16`, `i16`, `usize`, `isize` | 2 | `usize` is 16 bits |
| pointers, references, `fn` | 2 | |
| `&[T]`, `&str` | 4 | pointer and length |
| `u32`, `i32` | 4 | multiplication/division in the runtime |
| `u64`, `i64` | 8 | |
| `u128`, `i128` | 16 | add, subtract, logic, compare; no `*` `/` |
| `char` | 4 | avoid: use `u8` and ASCII |
| `Option<&T>`, `Option<fn()>` | 2 | null is `None` |

Everything has alignment 1. Struct fields may be reordered by rustc as usual; use `#[repr(C)]` when the layout matters (hardware registers, assembly).

### What does not work (yet)

| Feature | Instead |
|---|---|
| recursion | loops; the compiler reports the cycle |
| `dyn Trait` (vtables) | generics, enums, `fn` pointers |
| `f32`, `f64` | fixed point (`i16` with 8.8) |
| `alloc` (`Vec`, `String`, `Box`) | arrays, `c64::fmt::Buf` |

| Feature | Instead |
|---|---|
| `core::fmt` with arguments (`write!(f, "{}", x)`, `panic!("{}", x)`) | `c64::print!`, `c64::write!`, `panic!("text")` |
| `ptr::copy` (overlapping copies, memmove) | `c64::mem::copy` (copies upwards) |
| `u128` multiplication/division | |
| `asm!` | assembly functions (chapter 7) |
| threads, atomics | interrupts with `c64::irq`; `sys::without_interrupts` |

Recursion is not possible because every function has a static frame (see 5.3). A recursive program is rejected:

```
error: rust64: recursion is not supported (static frames): mygame::walk -> mygame::walk
```

## Panics

The target uses `panic = "immediate-abort"`: a panic turns the border red and returns to BASIC. There is no message. In debug builds overflow and `ub_checks` (pointer alignment, `get_unchecked` bounds, ...) are checked; release builds keep bounds checks but drop the others.

## Strings and PETSCII

String literals in the source are ASCII (UTF-8). The C64 uses PETSCII and its own screen codes:

- `c64::print!` / `println!` and `text::print` convert ASCII to PETSCII: `a-z` and `A-Z` both show as capitals in the default upper case/graphics character set; after `text::lowercase()` they show as written. `\n` becomes RETURN.
- `c64::p!("HELLO")` converts at compile time to a `&'static [u8; N]` of PETSCII for the upper case set; `c64::pm!` does it for the lower/upper case set. Use them for file names, KERNAL calls and `text::print_petscii`.
- `text::put_str` writes screen codes directly into screen memory (with a colour, no cursor), `text::screen_code` converts one character.
- `b'A'` and `b"..."` stay ASCII bytes.

# 5. Memory and the runtime

## Memory map

| Address | Contents |
|---|---|
| `$0002–$008F` | zero page: `#[c64::zp]` statics, then the frames of the most-called functions (saved at start, restored at exit) |
| `$00F9–$00FE` | the runtime's pointer registers (`zp_p0` - `zp_p2`) |
| `$0400–$07FF` | text screen (default) |
| `$0801` | BASIC line `SYS 2061`, then the runtime and the program |
| after the code | constant data and statics with initial values |
| after the data | BSS (not in the file): runtime buffers, static frames |
| up to `$9FFF` | the end: the program, its data and BSS must stay below |
| `$A000–$BFFF` | BASIC ROM (RAM after `hw::bank_out_basic()`) |
| `$C000–$CFFF` | free RAM |
| `$D000–$DFFF` | I/O: VIC-II, SID, colour RAM, CIAs |
| `$E000–$FFFF` | KERNAL ROM |

`#[c64::at]` statics with data are part of the file at their address (chapter 6), and the linker checks that the program ends below every placed static in RAM. Using `c64::gfx` places its colour matrix at `$4000`: then the program and its BSS must end below `$4000`.

## Start and exit

The program starts like a BASIC program (`RUN`). The runtime

1. saves the zero page `$02–$8F` and clears it,
2. saves the stack pointer, the IRQ vector (`$0314`), the CPU port (`$01`) and the VIC registers `$D011`, `$D015`, `$D016`, `$D018`, `$D020`, `$D021`, and CIA 2's port A (the VIC bank),
3. initializes the zero-page statics,
4. calls `main`.

When `main` returns, or on `sys::exit()` or a panic, it restores the interrupt setup (CIA timer IRQ on, raster IRQ off), the memory configuration, the VIC mode and bank, silences the SID and restores the zero page, so BASIC continues normally. The border and background colours are not restored, so the program's last screen stays readable (and the tests can read the border colour).

## Static frames

There is no stack for data: every function gets a fixed block of memory (its frame) in the BSS for its parameters, locals and temporaries. A call copies the arguments into the callee's frame, `jsr`s, and copies the result back. `&local` is an ordinary 16-bit address.

This is fast on the 6510, which has only 256 bytes of hardware stack and no stack-relative addressing, but it has two consequences:

- **No recursion**: a function must not run twice at the same time.
- **Interrupts**: a handler must not run a function that the main program is in the middle of. rust64 solves this by compiling every function that is reached through a function pointer (interrupt handlers are `fn()` values) a second time, with its own frames. Listings call these copies `(pointer copy)`. Statics are shared as usual: use `sys::without_interrupts` when main and a handler change the same data.

Values live in memory, not in registers: each operation loads, computes and stores byte by byte. The code is correct but not yet small or fast (see chapter 13).

## Runtime registers

The arithmetic helpers (`rt_mul`, `rt_udiv`, `rt_sdiv`, `rt_memcpy`, ...) use registers in the BSS: `rt_a`, `rt_b`, `rt_r`, `rt_t` (8 bytes each), `rt_n`, `rt_cnt`; calls through pointers use `rt_args` (32 bytes), `rt_ret` (16 bytes) and `rt_jvec`. The interrupt dispatcher saves them (`rt_ctx_save`), but only if the code reached by handlers uses them; otherwise it saves only the six zero-page pointer bytes. Only the helpers a program uses are part of it.

## Zero page

The zero page `$02–$8F` (BASIC's, free while the program runs) holds the `#[c64::zp]` statics and, in what is left, the frames at the bottom of the frame overlay: the leaf functions first. Machine code that you call (`sys::jsr`, a SID tune's player) must not use these addresses for its own data; `$FB–$FE`, the usual choice of such code, are the runtime's scratch pointers and may be overwritten by it between calls.

## 6. Attributes and macros

### `#[c64::main]` — the entry point

```
#[c64::main]
fn main() { ... }
```

The function takes no arguments and returns `()`. (It generates an `extern "C" fn` exported as `main`; you can also write that yourself.)

### `#[c64::at(ADDR)]` — fixed address

```
// loaded at $2000 together with the program
#[c64::at(0x2000)]
static SPRITE_DATA: [u8; 128] = *include_bytes!("sprites.bin");

// only placed: the memory is not initialized and not part of the file
#[c64::at(0xc000)]
static mut BUFFER: MaybeUninit<[u8; 1024]> = MaybeUninit::uninit();
```

- A static **with data** becomes part of the program file at that address (`$0801` or above, and above the program: the file is padded up to it).
- A static initialized with `MaybeUninit::uninit()` only gets the address. If it is in RAM above `$0801`, the build fails when the program and its BSS grow into it: `rust64: the program overlaps BUFFER at $c000`.
- Two placed statics must not overlap.

Hardware registers are better reached through `c64::hw` (it uses pointers to the fixed addresses), but `#[c64::at(0xd020)] static mut BORDER: MaybeUninit<u8>` works too.

### `#[c64::zp]` — zero page

```
#[c64::zp]
static mut PTR: u16 = 0x0400;
```

The static goes to the zero page `$02–$8F` (fast, and usable as a `(zp),y` pointer from assembly). It is initialized at start; it must not contain references. The zero page has 142 bytes for these statics.

### `include_bytes!` — embed a file

This is plain Rust: `static DATA: [u8; N] = *include_bytes!("file.bin");` puts the bytes into the program's data, with `#[c64::at]` at an address. A `&'static [u8]` works too: `static PIC: &[u8] = include_bytes!("pic.kla");`.

### `c64::sid!` — a SID tune

```
c64::sid!(MUSIC, "music.sid");          // path relative to Cargo.toml

MUSIC.start(MUSIC.start_song);          // init + play every frame
MUSIC.stop();
```

The macro reads the PSID/RSID header at compile time, places the tune's data at its load address and defines `static MUSIC: c64::sound::Tune` with `init`, `play`, `songs` and `start_song` (0-based). Tunes without a play address (that install their own interrupt) are rejected; so are tunes that load below `$0801`.

### `c64::p!` / `c64::pm!` — PETSCII literals

```
kernal::setnam(c64::p!("HIGHSCORE"));
text::print_petscii(c64::pm!("Mixed Case"));
```

### `c64::rat_source!` — include assembly

See chapter 7.

## Exported statics and functions

`#[unsafe(no_mangle)]` (or `#[unsafe(export_name = "...")]`) on a static or a function makes its symbol visible to assembly code, even when no Rust code uses it:

```
#[unsafe(no_mangle)]
static mut player_x: u8 = 0;          // assembly: lda player_x
```

Exported functions are called with the function pointer convention (arguments in `rt_args`, result in `rt_ret`).

## 7. Assembly functions

Routines written in rat assembly are declared as `extern "C"` functions and included with `c64::rat_source!`:

```
c64::rat_source!(DOUBLE, "double.rat");    // relative to this .rs file

unsafe extern "C" {
    fn double(v: u16) -> u16;
}

let x = unsafe { double(21) };             // 42
```

`src/double.rat`:

```
; double(v: u16) -> u16: v * 2
double:
        asl double__args
        rol double__args+1
        lda double__args
        ldx double__args+1
        rts
```

The macro creates a byte-array static in the link section `rat`; the backend includes the text of all such statics of the program and its dependencies. Labels starting with `@` are local to the previous label (rat syntax).

### Calling convention

| What | Where |
|---|---|
| all argument bytes, in order | `<name>__args` (defined by the backend) |
| the first three argument bytes | also in A, X, Y |
| a result of 1-3 bytes | A, X, Y (low byte in A) |
| a larger result | `<name>__res` |

Arguments are laid out one after the other: `fn f(a: u8, p: *const u8, n: u16)` has `a` at `f__args`, `p` at `f__args+1`, `n` at `f__args+3`. Structs as arguments or results need `#[repr(C)]`. A `&[u8]` is 4 bytes: pointer, then length.

The routine may use A, X, Y, the zero-page pointers `zp_p0` (`$FB`), `zp_p1` (`$FD`), `zp_p2` (`$F9`) and the runtime registers. It must end with `rts` (or `jmp` to another routine). Useful runtime labels:

| Label | Use |
|---|---|
| `rt_memcpy` | copy `rt_n` bytes from `(zp_p0)` to `(zp_p1)` |
| `rt_memset` | set `rt_n` bytes at `(zp_p1)` to A |
| `rt_exit` | return to BASIC |
| `rt_panic` | red border, return to BASIC |
| `rt_jsrind` | `jmp (rt_jvec)` |

A KERNAL routine can be declared directly when its registers fit the convention (`c64_chrout = $ffd2` in the library's `kernal.rat`). Look at `lib/c64/src/rat/*.rat` for more examples.

## 8. Library reference

The `c64` crate has these modules. Everything that touches the hardware is safe to call; only raw memory access (`mem::bytes`, `sys::jsr`) is `unsafe`.

### c64::hw — hardware registers

```
use c64::hw::{self, Color};

hw::vic().border.write(Color::Green);          // $d020
hw::vic().background[0].write(Color::Black);   // $d021
let line = hw::vic().raster.read();             // $d012
hw::sid().voice[0].freq.write(7493);
hw::cia1().pra.read();
```

| Item | Description |
|---|---|
| `vic()`, `sid()`, `cia1()`, `cia2()` | the chips as `#[repr(C)]` register structs |
| `Vic` | `sprite[8].{x,y}`, `sprite_x_msb`, `control1`, `raster`, `sprite_enable`, `control2`, `memory`, `irq_status`, `irq_enable`, `sprite_priority`, `sprite_multicolor`, `sprite_expand_x/_y`, `sprite_collision`, `sprite_bg_collision`, `border`, `background[4]`, `sprite_mc[2]`, `sprite_color[8]` |
| `Sid` | `voice[3].{freq, pulse, control, attack_decay, sustain_release}`, `cutoff`, `resonance`, `mode_volume`, `pot_x/_y`, `osc3`, `env3` |
| `Cia` | `pra`, `prb`, `ddra`, `ddrb`, `timer_a/_b`, `tod_*`, `sdr`, `icr`, `cra`, `crb` |
| `Volatile<T>` | `read()`, `write(v)`, `modify(f)`; for `u8` also `set_bits`, `clear_bits` |
| `ColorReg` | `read() -> Color`, `write(Color)` (colour registers) |
| `Color` | the 16 colours; `Color::from_u8(n)`, `c as u8` |
| `screen()`, `color_ram()`, `sprite_pointers()` | `[Volatile<u8>; N]` at `$0400`, `$D800`, `$07F8` |
| `cpu_port()` | `$01`, with `ALL_ROMS`, `NO_BASIC`, `IO_ONLY` |
| `raster_line()` | the 9-bit raster line (0-311) |
| `wait_raster(line)` | wait until the beam reaches `line` (0-255) in the next frame |
| `wait_frame()` | wait for the bottom of the screen (line 251) |
| `set_vic_bank(n)` | VIC bank 0-3 (`$0000`, `$4000`, `$8000`, `$C000`) |
| `set_screen(screen, charset)` | `$D018` in units of 1K and 2K |
| `bank_out_basic()` | RAM at `$A000–$BFFF` |

### c64::kernal — KERNAL ROM

Thin wrappers; routines that report errors return `Result<_, Error>` (`Error::FILE_NOT_FOUND`, `Error::DEVICE_NOT_PRESENT`, …).

| Function | KERNAL |
|---|---|
| `chrout(c)`, `chrin()`, `getin()` | `$FFD2`, `$FFCF`, `$FFE4` |
| `scnkey()`, `stop()` | `$FF9F`, `$FFE1` |
| `plot(row, col)`, `cursor() -> (row, col)` | `$FFF0` |
| `setlfs(lfn, dev, sa)`, `setnam(name)` | `$FFBA`, `$FFBD` |
| `open()`, `close(lfn)`, `chkin(lfn)`, `chkout(lfn)`, `clrchn()`, `clall()` | `$FFC0` … |
| `readst()` | `$FFB7` |
| `load(verify, addr) -> Result<end>`, `save(start, end)` | `$FFD5`, `$FFD8` |
| `rdtim() -> u32`, `settim(t)`, `jiffies() -> u16` | the 1/60 s clock |

Constants: `RETURN`, `CLEAR`, `HOME`, `UP`, `DOWN`, `LEFT`, `RIGHT`, `REVERSE_ON`, `REVERSE_OFF`, `LOWERCASE`, `UPPERCASE`, `DELETE`.

## c64::text — the character screen

| Function | Description |
|---|---|
| `print(s)`, `println(s)` | print ASCII at the cursor (PETSCII conversion) |
| `print_petscii(bytes)` | print PETSCII bytes unchanged |
| `clear()`, `locate(col, row)` | clear the screen, move the cursor |
| `set_color(c)`, `colors(border, bg)` | text colour; border and background |
| `lowercase()`, `uppercase()` | switch the character set |
| `put_char(col, row, code, color)` | write a screen code directly |
| `put_str(col, row, s, color)` | write a string directly (no cursor) |
| `fill(code, color)`, `fill_rect(col, row, w, h, code, color)` | fill the screen or a rectangle |
| `scroll_up(code, color)` | move everything up one line |
| `set_screen_addr(addr)`, `set_charset(addr)` | another screen / character set |
| `petscii(c)`, `screen_code(c)`, `petscii_to_screen(c)` | conversions |
| `Console` | the screen as a `fmt::Write` |
| `WIDTH`, `HEIGHT` | 40, 25 |

## c64::fmt — formatting

```
c64::println!("x=", x, " y=", y);
c64::print!(Hex(0xd020u16), " ", Bin(5u8));     // D020 00000101
let mut b = c64::fmt::Buf::<16>::new();
c64::write!(b, "score ", score);
text::put_str(0, 0, core::str::from_utf8(b.as_bytes()).unwrap_or(""), Color::White);
```

| Item | Description |
|---|---|
| `trait Write` | `write_byte`, `write_bytes`, `write_str` |
| `trait Display` | `fmt<W: Write>(&self, w)`, for `str`, `[u8]`, `bool`, `char`, all integers, `&T` |

| Item | Description |
|---|---|
| `Hex(v)`, `Bin(v)` | all digits, for `u8`/`u16`/`u32` (`Bin`: `u8`/`u16`) |
| `Buf<N>` | a fixed text buffer: `as_bytes`, `len`, `clear` |
| `write_u16`, `write_u32` | decimal without division |
| `print!`, `println!`, `write!` | comma-separated values |

Implement `Display` for your own types:

```
struct Pos(u8, u8);
impl c64::fmt::Display for Pos {
    fn fmt<W: c64::fmt::Write>(&self, w: &mut W) {
        c64::write!(*w, self.0, ',', self.1);
    }
}
```

## c64::sprite — hardware sprites

```
use c64::sprite::{self, Sprite};
sprite::load(13, &BALL);          // image block 13 = $0340
let s = Sprite(0);
s.set_image(13);
s.set_color(Color::Yellow);
s.move_to(160, 120);
s.show();
```

| Item | Description |
|---|---|
| `load(block, data)` | copy an image (up to 64 bytes) to `block * 64` |
| `Sprite(n)` | `set_image`, `show`, `hide`, `move_to(x: u16, y: u8)`, `x()`, `y()`, `set_color`, `set_multicolor`, `expand(x, y)`, `behind`, `hit(mask)` |
| `hide_all()`, `shared_colors(c1, c2)` | |
| `collisions()`, `bg_collisions()` | collision bits (cleared on read) |
| `set_pointer_addr(addr)` | after moving the screen |

The visible screen starts at x = 24, y = 50.

## c64::gfx — bitmap graphics

```
let g = gfx::hires(Color::White, Color::Black);    // Bitmap<Hires>
g.line(0, 0, 319, 199);
g.circle(160, 100, 50);
gfx::text();
```

| Item | Description |
|---|---|
| `hires(fg, bg) -> Bitmap<Hires>` | 320x200, clears the bitmap |
| `multicolor(bg, c1, c2, c3) -> Bitmap<Multicolor>` | 160x200, pens 0-3 |

| Item | Description |
|---|---|
| `show_koala(pic) -> Option<Bitmap<Multicolor>>` | a Koala picture (10003 bytes with load address) |
| `text()` | back to the text screen |
| `Bitmap<M>` | `pen`, `clear`, `plot(x, y)`, `line`, `rect`, `fill_rect`, `circle` |
| `Bitmap<Hires>` | also `point(x, y)`, `set_colors`, `set_cell_colors(col, row, fg, bg)` |
| `BITMAP`, `MATRIX` | `$6000`, `$4000` (VIC bank 1) |

Coordinates are `i16`; pixels outside the screen are skipped. The mode is part of the type, so `point` only exists for hires bitmaps.

## c64::sound — the SID

```
use c64::sound::{self, Note, Voice};
sound::volume(15);
let v = Voice(0);
v.adsr(0, 9, 0, 0);
v.note_on(Note::C4, sound::PULSE);
v.note_off();
```

| Item | Description |
|---|---|
| `Voice(n)` | `freq`, `pulse_width`, `adsr(a, d, s, r)`, `play(wave)`, `note_on(note, wave)`, `note_off()` |
| `Note(n)` | 0 = C0 … 95 = B7; `Note::new(octave, semitone)`, constants `C3` … `C5` (`CS4` = C#4), `freq()` (PAL) |
| `reset()`, `volume(v)`, `filter(cutoff, resonance, voices, mode)` | |
| waveforms | `TRIANGLE`, `SAWTOOTH`, `PULSE`, `NOISE`, bits `GATE`, `SYNC`, `RING`, `TEST` |
| filter | `LOW_PASS`, `BAND_PASS`, `HIGH_PASS`, `VOICE3_OFF`, `FILTER_VOICE1..3` |
| `Tune` | from `c64::sid!`: `start(song)`, `stop()`, fields `init`, `play`, `songs`, `start_song` |

The SID's registers are write-only; the module keeps copies of the control and volume registers.

## c64::irq — raster interrupts

```
fn top() { hw::vic().border.write(Color::Blue); }
fn bottom() { hw::vic().border.write(Color::Black); }

irq::add(50, top);          // at raster line 50
irq::add(200, bottom);      // ascending lines
irq::stop();
```

| Function | Description |
|---|---|
| `add(line, f: fn())` | install a handler (up to `MAX_HANDLERS` = 8) |

| Function | Description |
|---|---|
| `on_frame(f)` | once per frame at line 251 |
| `stop()` | remove all, restore the KERNAL interrupt |
| `count()` | installed handlers |
| `critical_section(f)` | run `f` with interrupts off |

Handlers run in order; after the last one of each frame the KERNAL's keyboard scan and jiffy clock run. The CIA timer interrupt is off while handlers are installed. If a handler runs past the next handler's line, that one follows immediately. Handlers can call any function (they get their own copies, see 5.3); keep them short.

## c64::input — joystick and keyboard

| Item | Description |
|---|---|
| `joy(Port::One / Port::Two) -> Joy` | `up()`, `down()`, `left()`, `right()`, `fire()`, `idle()` |
| `key() -> Option<u8>` | next key (PETSCII) from the buffer |
| `wait_key() -> u8` | wait for a key |
| `pressed(row, col)` | a key of the keyboard matrix |

## c64::dos — disk drives

```
let end = dos::load("LEVEL1", 8, 0xc000)?;
let mut f = dos::File::open(2, 8, 2, "SCORES,S,W")?;
f.write_str("12345")?;
drop(f);                                        // closes the file
dos::command(8, "S0:OLDFILE")?;
for e in dos::Dir::open(2, 8)? { ... }
```

| Item | Description |
|---|---|
| `load(name, dev, addr)`, `load_abs(name, dev)` | load to `addr` / to the file's address; returns the end |
| `save(name, dev, start, end)` | save `start..end` |
| `command(dev, cmd)`, `scratch(dev, name)`, `rename(dev, from, to)` | DOS commands |
| `status(dev, buf) -> Option<(len, code)>` | the drive status (`00, OK,00,00`) |
| `File::open(lfn, dev, sa, name)` | `get()`, `read(buf)`, `eof()`, `write(bytes)`, `write_str(s)`; closed on drop |
| `Dir::open(lfn, dev)` | an `Iterator<Item = DirEntry>`; `header`, `free()` |
| `DirEntry` | `blocks`, `name()` (PETSCII), `kind` (`PRG`, `SEQ`, …) |

File names are ASCII and converted to PETSCII capitals.

## c64::mem — raw memory

| Function | Description |
|---|---|
| `peek(a)`, `poke(a, v)`, `peek_word(a)`, `poke_word(a, v)` | volatile accesses |

| Function | Description |
|---|---|
| `copy(dst, src, n)` | fast copy, front to back |
| `fill(dst, n, v)` | fast fill |
| `unsafe bytes(a, n) -> &'static mut [u8]` | memory as a slice |
| `addr(a) -> *mut u8` | a pointer to an address |

## c64::sys — program control

| Item | Description |
|---|---|
| `exit() -> !` | back to BASIC from anywhere |
| `panic() -> !` | red border, back to BASIC |
| `sei()`, `cli()`, `without_interrupts(f)` | the interrupt flag |
| `unsafe jsr(addr, Regs) -> Regs` | call machine code with A/X/Y, get A/X/Y/P back ( `carry()` ) |

# 9. Cookbook

All recipes compile with the current rust64. Each is a complete `src/main.rs` or a part of one (with the `#![no_std]`, `#![no_main]` header from chapter 2).

## A moving sprite with the joystick

`examples/sprites`: a ball moves with the joystick in port 2; fire changes its colour; a raster interrupt paints a band behind the title.

```rust
#![no_std]
#![no_main]

use c64::hw::{self, Color};
use c64::input::{self, Port};
use c64::sprite::{self, Sprite};
use c64::{irq, text};

/// A 24x21 ball, 3 bytes per line.
static BALL: [u8; 63] = [
    0x00, 0x7e, 0x00, 0x03, 0xff, 0xc0, 0x07, 0xff, 0xe0, 0x1f, 0xff, 0xf8, 0x1f, 0xff, 0xf8,
    0x3f, 0xff, 0xfc, 0x7f, 0xff, 0xfe, 0x7f, 0xff, 0xfe, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0x7f, 0xff, 0xfe, 0x7f, 0xff, 0xfe,
    0x3f, 0xff, 0xfc, 0x1f, 0xff, 0xf8, 0x1f, 0xff, 0xf8, 0x07, 0xff, 0xe0, 0x03, 0xff, 0xc0,
    0x00, 0x7e, 0x00,
];

fn band_on() {
    hw::vic().background[0].write(Color::Blue);
}

fn band_off() {
    hw::vic().background[0].write(Color::Black);
}

#[c64::main]
fn main() {
    text::colors(Color::Black, Color::Black);
    text::clear();
    text::put_str(11, 1, "rust64 sprite demo", Color::White);
    text::put_str(8, 23, "joystick 2 - fire: colour", Color::Grey);

    sprite::load(13, &BALL);
    let ball = Sprite(0);
    ball.set_image(13);
    ball.set_color(Color::Yellow);
    ball.show();

    irq::add(50, band_on);
    irq::add(66, band_off);

    let (mut x, mut y) = (160u16, 140u8);
    let mut color = Color::Yellow as u8;
    let mut fire_held = false;
    loop {
        hw::wait_frame();
        let j = input::joy(Port::Two);
        if j.left() && x > 24 { x -= 2; }
        if j.right() && x < 320 { x += 2; }
        if j.up() && y > 50 { y -= 2; }
        if j.down() && y < 229 { y += 2; }
        let fire = j.fire();
        if fire && !fire_held {
            color = (color + 1) & 15;
            if color == Color::Black as u8 {
                color = Color::White as u8;
            }
            ball.set_color(Color::from_u8(color));
        }
        fire_held = fire;
        ball.move_to(x, y);
    }
}
```

## Raster bars

Four handlers on consecutive lines change the border colour:

```
use c64::hw::{self, Color};
use c64::{input, irq};

static COLORS: [Color; 4] = [Color::Red, Color::Orange, Color::Yellow, Color::Black];
static mut BAR: u8 = 0;

fn bar() {
    unsafe {
        hw::vic().border.write(COLORS[BAR as usize]);
        BAR = (BAR + 1) & 3;
    }
}

fn raster_bars() {
    for line in [60, 70, 80, 90] {
        irq::add(line, bar);
    }
    input::wait_key();
    irq::stop();
}
```

A handler costs a few raster lines of overhead (more if its code uses the runtime's arithmetic helpers), so keep the lines some distance apart.

## Drawing on the bitmap

`examples/drawing`:

```
use c64::gfx;
use c64::hw::{self, Color};
use c64::input;

#[c64::main]
fn main() {
    hw::vic().border.write(Color::Black);
    let g = gfx::hires(Color::LightGreen, Color::Black);
    for i in (0..320i16).step_by(16) {
        g.line(i, 0, 319 - i, 199);
    }
    g.rect(40, 30, 240, 140);
    for r in (10..=60i16).step_by(10) {
        g.circle(160, 100, r);
    }
    for c in 0..40u8 {
        g.set_cell_colors(c, 24, Color::from_u8(c & 15 | 1), Color::Black);
    }
    input::wait_key();
    gfx::text();
}
```

## Playing notes

`examples/music` plays a melody on voice 1 and a bass on voice 2, timed with the jiffy clock:

```rust
use c64::hw::{self, Color};
use c64::sound::{self, Note, Voice};
use c64::{kernal, text};

/// A note: pitch (`None` = rest) and length in jiffies.
#[derive(Clone, Copy)]
struct Tone(Option<Note>, u8);

const fn n(note: Note, len: u8) -> Tone {
    Tone(Some(note), len)
}

static MELODY: [Tone; 26] = [
    n(Note::E4, 15), n(Note::D4, 15), n(Note::C4, 15), n(Note::D4, 15),
    n(Note::E4, 15), n(Note::E4, 15), n(Note::E4, 30),
    n(Note::D4, 15), n(Note::D4, 15), n(Note::D4, 30),
    n(Note::E4, 15), n(Note::G4, 15), n(Note::G4, 30),
    n(Note::E4, 15), n(Note::D4, 15), n(Note::C4, 15), n(Note::D4, 15),
    n(Note::E4, 15), n(Note::E4, 15), n(Note::E4, 15), n(Note::E4, 15),
    n(Note::D4, 15), n(Note::D4, 15), n(Note::E4, 15), n(Note::D4, 15),
    n(Note::C4, 60),
];

static BASS: [Note; 4] = [Note::C3, Note::G3, Note::A3, Note::F3];

fn wait(jiffies: u8) {
    let start = kernal::jiffies();
    while kernal::jiffies().wrapping_sub(start) < jiffies as u16 {}
}

#[c64::main]
fn main() {
    text::colors(Color::Blue, Color::Black);
    text::clear();
    c64::println!("rust64 music");
    sound::reset();
    sound::volume(15);
    let (lead, bass) = (Voice(0), Voice(1));
    lead.adsr(0, 9, 4, 4);
    lead.pulse_width(0x800);
    bass.adsr(0, 4, 10, 6);
    for (i, t) in MELODY.iter().enumerate() {
        if i % 4 == 0 {
            bass.note_on(BASS[(i / 4) % 4], sound::TRIANGLE);
        }
        if let Some(note) = t.0 {
            lead.note_on(note, sound::PULSE);
            hw::vic().border.write(Color::from_u8(note.0));
        }
        wait(t.1 - 3);
        lead.note_off();
        wait(3);
    }
    bass.note_off();
    wait(60);
    sound::reset();
    c64::println!("done");
}
```

## Playing a SID tune

```
use c64::input;

c64::sid!(MUSIC, "music.sid");

#[c64::main]
fn main() {
    c64::println!("playing, press a key");
    MUSIC.start(MUSIC.start_song);
    input::wait_key();
    MUSIC.stop();
}
```

The tune's data is loaded at its own address, so the program must end below it. `start` calls the tune's init routine and its play routine once per frame from a raster interrupt.

## Saving and loading a high score

`dos::save` writes a memory range as a program file; `dos::load` reads it back to an address:

```
use c64::{dos, kernal};

static mut HIGH_SCORE: u16 = 0;

fn save_high_score() -> Result<(), kernal::Error> {
    let a = (&raw const HIGH_SCORE) as usize as u16;
    let _ = dos::scratch(8, "HIGHSCORE");       // replace an old file
    dos::save("HIGHSCORE", 8, a, a + 2)
}

fn load_high_score() {
    let a = (&raw mut HIGH_SCORE) as usize as u16;
    if dos::load("HIGHSCORE", 8, a).is_err() {
        unsafe { HIGH_SCORE = 0 };
    }
}
```

## Writing and reading a sequential file

```
use c64::dos::File;
use c64::{kernal, text};

fn write_log() -> Result<(), kernal::Error> {
    let mut f = File::open(2, 8, 2, "LOG,S,W")?;
    f.write_str("LEVEL 1 DONE\n")?;
    Ok(())                                  // f is closed here
}

fn read_log() {
    let Ok(mut f) = File::open(2, 8, 2, "LOG,S,R") else { return };
    let mut buf = [0u8; 40];
    let n = f.read(&mut buf);
    text::print_petscii(&buf[..n]);
}
```

Drive errors such as "file not found" are not KERNAL errors: read them with `dos::status(8, &mut buf)`, which returns the two-digit code (62 = file not found).

## Listing the directory

`examples/dir`:

```
use c64::dos::{self, Dir};
use c64::text;

#[c64::main]
fn main() {
    let mut d = match Dir::open(2, 8) {
        Ok(d) => d,
        Err(e) => {
            c64::println!("cannot open directory, error ", e.0);
            return;
        }
    };
    c64::print!("disk: ");
    text::print_petscii(d.header.name());
    c64::println!();
    for e in &mut d {
        c64::print!(e.blocks, " ");
        text::print_petscii(e.name());
        c64::print!(" ");
        text::print_petscii(&e.kind);
        c64::println!();
    }
    c64::println!(d.free(), " blocks free");
    drop(d);

    let mut buf = [0u8; 40];
    if let Some((n, _)) = dos::status(8, &mut buf) {
        text::print_petscii(&buf[..n]);
        c64::println!();
    }
}
```

Build it as a disk image and run it from there, so the drive has something to list:

```
cargo c64 run --release -o dir.d64
```

## A custom character set

Copy the ROM character set into RAM, change a character, and point the VIC at the copy (the program must end below `$3000`):

```
use c64::{hw, mem, sys, text};

fn custom_charset() {
    sys::without_interrupts(|| {
        hw::cpu_port().write(0x33);          // character ROM at $D000
        mem::copy(0x3000, 0xd000, 2048);
        hw::cpu_port().write(hw::ALL_ROMS);
    });
    // character 0 ("@"): a smiley
    let smiley = [0x3c, 0x42, 0xa5, 0x81, 0xa5, 0x99, 0x42, 0x3c];
    for (i, &b) in smiley.iter().enumerate() {
        mem::poke(0x3000 + i as u16, b);
    }
    text::set_charset(0x3000);
}
```

### Callbacks with function pointers

```
fn apply(f: fn(u8) -> u8, xs: &mut [u8]) {
    for x in xs {
        *x = f(*x);
    }
}

let mut xs = [1u8, 2, 3];
apply(|x| x * 3, &mut xs);                  // [3, 6, 9]
```

Generic functions (`fn apply<F: Fn(u8) -> u8>(f: F, ...)`) are faster: the call is direct and can be inlined. Function pointers are for tables of handlers and for interrupts.

### Zero page and fixed addresses

```
use core::mem::MaybeUninit;

#[c64::zp]
static mut FRAME: u8 = 0;                   // somewhere in $02-$8F

#[c64::at(0xc000)]
static mut BUFFER: MaybeUninit<[u8; 256]> = MaybeUninit::uninit();

unsafe {
    FRAME += 1;
    let p = (&raw mut BUFFER).cast::<u8>();  // 0xc000
    *p = 42;
}
```

### Formatting into a buffer

```
use c64::fmt::{Bin, Buf, Hex};
use c64::hw::Color;
use c64::text;

c64::println!("x=", 10u8, " hex ", Hex(0xd020u16), " bin ", Bin(5u8), " neg ", -3i16);
// X=10 HEX D020 BIN 00000101 NEG -3

let mut b = Buf::<16>::new();
c64::write!(b, "score ", 1200u16);
text::put_str(0, 24, core::str::from_utf8(b.as_bytes()).unwrap_or(""), Color::Yellow);
```

### Calling assembly

See chapter 7 for the `double` example with its own `.rat` file. For a routine that already exists in ROM or memory, `sys::jsr` passes A, X, Y and returns the registers and flags afterwards:

```
use c64::sys::{self, Regs};

// GETIN ($FFE4): A = the next key, 0 if none
let r = unsafe { sys::jsr(0xffe4, Regs::default()) };
if r.a != 0 {
    c64::println!("key ", r.a);
}
```

`sys::jsr` cannot set the carry flag before the call; routines that read it (like PLOT) need a wrapper in assembly.

## 10. Inside the backend

```
.rs ──► rustc (pinned nightly, target mos-c64.json: 16-bit pointers, os = c64)
        │  parse, types, borrow check, monomorphization, MIR optimizations
        ▼
 rustc_codegen_rust64   (-Zcodegen-backend)
        │  library crates: only metadata with MIR (-Zalways-encode-mir)
        │  the program: walk the MIR from `main` through all crates
        │  static frames, recursion check, pointer copies, data, zero page
        ▼
 rat assembly ──► rat-asm (linked in) ──► .prg + .d64
```

The target is a JSON specification based on `msp430` (rustc has no 6502 architecture; msp430 is 16-bit little endian, and LLVM is never called), with `target_os = "c64"`, 16-bit pointers, alignment 1 everywhere, `panic-strategy = immediate-abort` and the file suffix `.prg`.

The backend compiles the whole program at once, like Miri: starting at `main` it fetches the MIR of every function it reaches (`tcx.instance_mir`), in any crate, and translates it. This is why library crates need `-Zalways-encode-mir`, and why unused code costs nothing.

Details worth knowing:

- Constants and `ub_checks` are folded: blocks behind a constant condition are not compiled.
- Diverging functions of `core` (`panic_fmt`, `slice_index_fail`, ...) become `jmp rt_panic` without being compiled; they would pull in `core::fmt`.
- Statics are evaluated by rustc (`const` evaluation) and emitted as bytes; pointers inside them become `.word` relocations.
- The crate scan finds `#[link_section = "rat"]` statics (assembly), `extern "C"` declarations (argument areas) and exported symbols in the program and every dependency except the standard library.

### What the code looks like

`examples/border` writes 5 to `$D020`:

```rust
#[unsafe(no_mangle)]
pub extern "C" fn main() {
    unsafe { core::ptr::write_volatile(0xd020 as *mut u8, 5) };
}
```

```
; fn main
F0:
F0_b0:
        lda #$05
        sta $d020
F0_b1:
        rts
```

Labels: `F<n>` is function n, `F<n>_b<k>` its basic block k, `F<n>f` its frame, `F<n>v<i>` the storage of its local i, `F<n>t` its temporaries, `L<n>_<i>` its result (0) and arguments (1...), `A<n>` constant data, `S<n>` statics, `F<n>_ind` the pointer entry of a function.

Every value still lives in memory (there is no register allocation), but the backend avoids most of the cost of that:

- **Known addresses.** A pointer that always holds the same address (hardware registers, statics, `&local`) is used as a constant: accesses are direct (`sta $d020`), and the pointer itself is never stored.
- **Views instead of copies.** A temporary that is only a copy, a truncation, a zero-extension, a byte-preserving cast or a tuple/struct built from other values gets no storage: its users read (or dereference) the source bytes directly, also across branches, as long as the source cannot change before a use. Zero-extended high bytes become `#0`, which also shortens the 16-bit arithmetic that follows.
- **Compare and branch.** A comparison that feeds a branch or a bounds check is not turned into a `bool`: `lda i / cmp #25 / bcc ok`. Constant high bytes are skipped, `x < 0` tests the sign bit.
- **Tables.** Arrays of up to 256 bytes at a known address are indexed with X: `lda table,x`.
- **Small arithmetic.** Shifts by a constant are unrolled in place; unsigned `* / %` by powers of two become shifts and masks; `x += 1` is `inc` (with `bne`/`inc` for 16 bits); zero bytes of a constant are not added; `and #$ff` and similar disappear.
- **Frame overlay and zero page.** Frames only contain the locals that are really stored, and functions that are never active at the same time share memory: a frame lies above the frames of everything the function calls, so leaves are at the bottom. Whatever fits into the free zero page goes there, the leaves (called most) first: shorter and faster instructions, and pointers there are dereferenced directly with `(ptr),y`. Function pointer targets (interrupt handlers) get their own region.
- **Control flow.** Blocks are laid out along the usual path, so most jumps fall through; jumps to jumps and to the next line are removed; `bcc skip / jmp target` becomes `bcs target` when the target is certainly in range.
- **Peephole.** Loads of a value that A, X or Y already holds are removed (never reads of numeric addresses, which may be I/O registers).
- **Pruning.** Only the runtime and library assembly routines that the program refers to are included: the smallest program is about 200 bytes.

A KERNAL call looks like this:

```
; fn c64::kernal::chrout
F3:
F3_b0:
        lda F3v1
        sta c64_chrout__args+0
        jsr c64_chrout          ; = $ffd2
F3_b1:
        rts
```

## Source layout

| Path | Contents |
|---|---|
| `crates/rustc_codegen_rust64/src/lib.rs` | the `CodegenBackend`: drives the build, writes `.prg`, `.rat`, `.d64` |
| `crates/rustc_codegen_rust64/src/codegen.rs` | MIR to 6510: whole program, frames, statements, calls, intrinsics, data |
| `crates/rustc_codegen_rust64/src/asm.rs` | operands (absolute, indirect) and byte-wise code emission |
| `crates/rustc_codegen_rust64/src/runtime.rat` | start/exit, panic, memcpy/memset/memcmp, mul/div, bit counts |
| `crates/rustc_codegen_rust64/src/runtime.rs` | the call into rat |
| `crates/cargo-c64` | the `cargo c64` command and the end-to-end test harness |
| `target-spec/mos-c64.json` | the target |

| Path | Contents |
| --- | --- |
| `lib/c64` | the C64 library (Rust and `src/rat/*.rat`) |
| `lib/c64-macros` | `#[c64::main]`, `#[c64::at]`, `#[c64::zp]`, `c64::sid!` |
| `examples/` | example programs |
| `tests/e2e/` | end-to-end test programs |
| `PLAN.md`, `docs/spike.md` | design and findings |

# 11. Testing

## End-to-end tests

Every package in `tests/e2e/` is a C64 program that checks itself and turns the border green (all checks passed) or red (a check failed or it panicked). The harness builds each one in debug and release with `cargo c64`, runs it in mimic-headless for 600 frames and looks at the border colour:

```
cargo test -p cargo-c64 -- --nocapture
```

```
ok   core (debug)
ok   core (release)
ok   gfx (debug)
...
```

| Test | Covers |
|---|---|
| `core` | the language: arithmetic of all widths, comparisons, loops, arrays, slices, structs, enums, `Option`, statics, strings, closures, iterators |
| `lib` | printing, cursor, formatting, memory, `sys::jsr`, assembly functions, hardware registers, input |
| `lib2` | function pointers, `at`/`zp` statics, exported statics, raster interrupts, SID tunes, sprites, sound, PETSCII literals |
| `gfx` | hires and multicolour drawing, cell colours, bitmap mode |

A test program writes the number of the first failing check to the screen, so the snapshot (`rust64-e2e-<name>-<profile>.png` in the temp directory) shows what went wrong. The tests need mimic-headless (`RUST64_MIMIC_HEADLESS` or `../mimic/target/release`) and the ROMs; without the emulator they are skipped.

A minimal test program:

```
#![no_std]
#![no_main]
use c64::hw::{self, Color};
use core::hint::black_box as bb;

static mut FAILS: u8 = 0;

fn check(ok: bool) {
    if !ok {
        unsafe { FAILS += 1 };
    }
}

#[c64::main]
fn main() {
    check(bb(6u8) * 7 == 42);
    check(c64::text::screen_code(b'A') == 1);
    let ok = unsafe { FAILS == 0 };
    hw::vic().border.write(if ok { Color::Green } else { Color::Red });
}
```

`black_box` keeps rustc from computing the checks at compile time.

## Host tests

The `c64` crate's logic (formatting, screen codes, drawing, memory) has unit tests that run on the PC with the stand-ins:

```
cd lib/c64
cargo test
```

## Debugging

- `cargo c64 asm` prints the generated assembly.
- mimic's debug server (`mimic-headless --debug`) has breakpoints, watchpoints, `peek` and tracing; load the labels with rat (`rat build x.rat --sym x.sym`) to see function names.
- `RUST64_TRACE=1` prints the MIR of every compiled function.
- Backend errors name the call path from `main` to the function.

## 12. Troubleshooting

| Message / symptom | Cause and fix |
|---|---|
| `no entry point` | mark the start function with `#[c64::main]` and use `#![no_main]` |
| `recursion is not supported (static frames): a -> b -> a` | rewrite as a loop |
| `dynamic dispatch is not supported yet` | `dyn Trait`: use generics or an enum |
| `unsizing ... is not supported yet (trait objects)` | a `&dyn` coercion; as above |
| `floating point is not supported yet` | use integers / fixed point |
| `program and its data do not fit below $a000` | the program is too big; build with `--release`, use smaller tables |
| `the program overlaps X at $4000` | a placed static (here `gfx`'s colour matrix) is inside the program: make the program smaller (release, `opt-level = "s"`, in debug `debug-assertions = false`) or place your data higher |
| `static X at $... has data, but the program file cannot load data there` | placed data below `$0801`; use `MaybeUninit::uninit()` |
| `the zero page is full` | fewer or smaller `#[c64::zp]` statics |
| `... takes N bytes of arguments; calls through pointers allow 32` | pass a reference instead of a large value |
| `rat failed on the generated assembly` | an undefined label in your rat code, or a branch too far; the listing is kept |
| red border right away | a panic: index out of bounds, `unwrap` on `None`, overflow in debug |
| the program hangs in a busy loop | do not compare the raster line for equality, use `hw::wait_raster`; debug code is slow |
| a handler makes the program crawl | the handler uses runtime arithmetic (multiplication, division): the dispatcher then saves all runtime registers, and the unoptimized code is slow; keep handlers simple |
| `error: could not create LLVM MCSubtargetInfo` | rustc ran without the backend: use `cargo c64`, not plain `cargo build` |
| `backend ... not found` | build `rustc_codegen_rust64` (same profile as `cargo-c64`) or set `RUST64_BACKEND` |

## 13. Limitations and roadmap

rust64 follows the phases in `PLAN.md`:

| Phase | Status |
|---|---|
| 0. Spike: target, empty backend, `build-std` | done |
| 1. Core backend: integers, control flow, calls, static frames | done |
| 2. Language breadth: enums, structs, slices, closures, iterators, panics | done |
| 3. Library I: `hw`, `mem`, `sys`, `kernal`, `text`, `fmt`, `input`, assembly, `at` / `zp`, PETSCII | done |
| 4. Library II: `sprite`, `gfx`, `sound`, `irq`, `dos`, function pointers, D64 | done |
| 5. Optimization: frame overlay, zero-page allocation, 8-bit narrowing, block layout, peephole, pruning (done); X/Y loops, register values, inlining (open) | mostly done |
| 6. Tooling: `cargo c64 new/test`, Rust source spans for rat errors, memory map, debugger labels, `c64::asm!` | |
| 7. Extensions: recursion on a software stack, `alloc`, `dyn Trait`, `f32`, cartridges | |

Code quality: the optimizations of chapter 10 made the example programs 48-76% smaller and up to 2.75 times as fast as the unoptimized backend (`hello`: 3123 to 962 bytes, `sprites`: 6335 to 2143, `drawing` finishes its picture in 167 instead of 460 frames, about 390 cycles per pixel). Values still live in memory and are handled byte by byte, so tight loops are slower than hand-written assembly. Still open: values in A/X/Y across statements, 8-bit loop counters in X/Y, inlining of small functions when the program is put together (rustc cannot inline trait methods called from generic code, such as the pixel routine of `gfx`). Release builds (`opt-level = "s"`) give the best code. Debug builds include rustc's runtime checks and are much bigger and slower.

Other open points:

- No `dyn Trait`, floats, heap, recursion, `core::fmt` with arguments.
- Recursion through function pointers is not detected.
- A pointer copy that the main program calls (through a function pointer) while an interrupt runs the same copy shares its frame.
- `MulWithOverflow` does not report overflow.
- `#[track_caller]` locations are dummies (panics have no message anyway).